

ESP32S3 ↔ AB5605B 串口通信协议

版本: V1.0

作者: 余清礼

日期: 2026-06-25

适用范围: ESP32S3 (上控) 通过 UART 控制 AB5605B (下控) 蓝牙音箱

修订历史

版本	日期	修改内容	作者
V1.0	2026-06-25	初始版本	余清礼

目录

- [协议约定](#)
- [帧格式](#)
- [命令字映射](#)
- [DP点定义](#)
- [命令详细定义](#)
- [通信流程示例](#)
- [与现有协议的兼容关系](#)
- [错误处理](#)
- [实施建议](#)

一、协议约定

参数	值
通讯方式	串行异步通讯 (UART)
接口电平	TTL / LVTTTL (3.3V)
传送方式	全双工
波特率	115200 bps (可协商降至 57600 / 19200 / 9600)
数据位	8 bit
停止位	1 bit
校验位	无 (帧级校验和保证完整性)
字节序	大端模式 (Big-Endian)
通信模式	主从问答 + 下控主动上报

角色定义

角色	设备	版本字	说明
上控 (Master)	ESP32S3	0x01	发起命令、控制设备、发起OTA
下控 (Slave)	AB5605B	0x03	执行命令、上报状态、接收OTA

二、帧格式

所有数据帧采用统一格式：

帧头	版本	命令字	数据长度	数据	校验和	帧尾
1 Byte	1 Byte	1 Byte	2 Bytes	N Bytes	1 Byte	1 Byte
0x55	0x01/3	CMD	LEN	PAYLOAD	SUM%256	0xAA

字段	字节数	说明
帧头	1	固定 0x55
版本	1	上控 0x01，下控 0x03
命令字	1	帧类型，详见第三章
数据长度	2	仅数据字段的字节数，大端模式
数据	N	实际负载，格式取决于命令字
校验和	1	从帧头到数据末尾按字节累加，对256取余
帧尾	1	固定 0xAA

校验和计算示例

帧：55 01 06 00 05 01 01 00 01 01 ?? AA
↑ 帧头为 0x55
$\begin{aligned} \text{checksum} &= (0x55 + 0x01 + 0x06 + 0x00 + 0x05 + 0x01 + 0x01 + 0x00 + 0x01 + 0x01) \\ &\% 256 \\ &= 0x65 \end{aligned}$

帧同步说明

- 连续字节流中通过 0x55 帧头定位起始位置
- 读取帧头后，读取5字节（版本+命令+长度），再读取N字节数据+1字节校验+1字节帧尾
- 0x55 出现在数据字段中**不转义**（因为有长度字段，不依赖帧头定界）

三、命令字映射

3.1 基础协议命令 (0x00 ~ 0x0F)

命令字	名称	方向	功能
0x00	心跳检测	双向	检测对方是否在线
0x01	查询设备信息	上控→下控	获取硬件版本、软件版本、产品名
0x02	系统控制	上控→下控	复位、进入Bootloader等
0x03	静音控制	上控→下控	★ 直接静音/取消静音（快速通道）
0x04	音量控制	上控→下控	★ 设置音量 (0~16)
0x06	DP命令下发	上控→下控	通用DP点设置，可批量
0x07	DP状态上报	下控→上控	下控主动上报或响应查询
0x08	DP状态查询	上控→下控	查询指定DP点当前值
0x09	查询设备信息(扩展)	上控→下控	返回结构化二进制信息

3.2 OTA升级命令 (0x0A ~ 0x0F)

命令字	名称	方向	功能
0x0A	OTA开始	上控→下控	发送文件大小+类型，协商块大小
0x0B	OTA数据	上控→下控	传输数据块（偏移+数据+XOR）
0x0C	OTA完成	上控→下控	发送CRC16，设备校验后应答

四、DP点定义 (AB5605B 音频系统)

继承MRK协议DP点框架，用于 0x06 命令下发 和 0x07 状态上报。

4.1 DP数据结构

单个DP单元格式：

DP ID	数据类型	数据长度	数据值
1 Byte	1 Byte	2 Bytes	N Bytes

4.2 数据类型枚举

编码	类型	数据长度	说明
0x01	bool	1	0x00 =假, 0x01 =真
0x02	value	4	有符号/无符号整数, 大端
0x03	string	变长	UTF-8字符串
0x04	enum	1	枚举值
0x05	bitmap	1/2/4	位图

4.3 AB5605B DP点列表

DP ID	功能名称	数据类型	功能属性	取值范围 / 说明
0x01	静音状态	bool	可下发可上报	0x00 :取消静音, 0x01 :静音
0x02	音量级别	value	可下发可上报	0~30
0x03	音频源	enum	只上报	0x00 :无, 0x01 :BT-A2DP, 0x02 :AUX
0x04	蓝牙连接状态	enum	只上报	0x00 :断开, 0x01 :连接中, 0x02 :已连接, 0x03 :播放中
0x10	硬件版本	string	只上报	格式: "x.x.x", 如 "1.0.0"
0x11	软件版本	string	只上报	格式: "x.x.x", 如 "1.0.0"
0x12	产品名称	string	只上报	如 "BT-BOX"
0x13	厂商名称	string	只上报	如 "MRK"
0x20	语音OTA状态	enum	只上报	0x00 :空闲, 0x01 :接收中, 0x02 :校验中, 0x03 :成功, 0x04 :失败
0x21	语音OTA进度	value	只上报	0~100 (百分比)
0x30	均衡器模式	enum	可下发可上报	0x00 :Normal, 0x01 :Pop, 0x02 :Rock, 0x03 :jazz, 0x04 :Classic
0x31	LED灯效	enum	可下发可上报	0x00 :关闭, 0x01 :常亮, 0x02 :呼吸, 0x03 :音乐律动

五、命令详细定义

5.1 心跳检测 (0x00)

上控→下控：

字段	值	说明
帧头	0x55	
版本	0x01	上控
命令	0x00	心跳
数据长度	0x0000	无数据
校验和	计算	
帧尾	0xAA	

完整帧：55 01 00 00 00 56 AA

下控→上控 (应答)：

字段	值	说明
帧头	0x55	
版本	0x03	下控
命令	0x00	心跳
数据长度	0x0001	1字节
数据	0x01	在线
校验和	计算	
帧尾	0xAA	

完整帧：55 03 00 00 01 01 5A AA

超时： 上控发送后等待 3s，未收到应答视为下控离线。

5.2 查询设备信息 (0x01)

上控→下控：

55 01 01 00 00 57 AA

下控→上控 (应答)：

数据部分为JSON字符串（UTF-8）：

```
{ "pn": "BT-BOX", "fn": "MRK", "hv": "1.0.0", "sv": "1.0.1" }
```

字段	说明	示例
pn	产品名称 (Product Name)	"BT-BOX"
fn	厂商名称 (Factory Name)	"MRK"
hv	硬件版本 (Hardware Version)	"1.0.0"
sv	软件版本 (Software Version)	"1.0.1"

完整帧示例 (JSON长度=55 = 0x0037)：

```
55 03 01 00 37 7B 22 70 6E 22 3A 22 42 54  ... 7D XX AA
      ↑ JSON字符串 ↑
```

5.3 系统控制 (0x02)

上控→下控：

```
55 01 02 00 01 [CTRL] XX AA
```

控制码 (CTRL)：

值	功能	说明
0x01	软复位	AB5605B 执行软复位
0x02	进入固件OTA模式	准备接收完整固件升级
0x03	进入语音OTA模式	准备接收语音文件升级

下控→上控：

```
55 03 02 00 01 [RESULT] XX AA
```

RESULT：

值	说明
0x00	成功
0x01	不支持
0x02	当前状态不允许

示例 — 请求软复位：

```
上控发送：55 01 02 00 01 01 5A AA
下控应答：55 03 02 00 01 00 5B AA    (成功后约200ms执行复位)
```

5.4 静音控制 (0x03) ★

上控→下控:

55 01 03 00 01 [STATE] XX AA

STATE:

值	说明
0x00	取消静音
0x01	静音

下控→上控:

55 03 03 00 01 [RESULT] XX AA

RESULT: 0x00=成功

示例 — 静音:

上控发送: 55 01 03 00 01 01 5B AA
下控应答: 55 03 03 00 01 00 5C AA

5.5 音量控制 (0x04) ★

上控→下控:

55 01 04 00 01 [VOL] XX AA

VOL: 0~30 (对应AB5605B系统音量)

下控→上控:

55 03 04 00 01 [RESULT] XX AA

RESULT:

值	说明
0x00	成功
0x01	超出范围

示例 — 设置音量15:

上控发送: 55 01 04 00 01 0F 6A AA
下控应答: 55 03 04 00 01 00 5D AA

5.6 DP命令下发 (0x06)

数据部分为连续多个DP单元：

上控发送：

55 01 06 [LEN_HI] [LEN_LO] [DP1单元] [DP2单元] ... [DPn单元] [CKSUM] AA

DP单元 = [DP_ID 1B][TYPE 1B][VAL_LEN 2B][VALUE NB]

示例1 — 单DP：设置静音 (DP 0x01)

55 01 06 00 05

01

01

00 01

01

[校验] AA

← 帧头+数据长度=5

← DP ID=1（静音状态）

← 类型=bool

← 值长度=1

← 值=0x01（静音）

打包：55 01 06 00 05 01 01 00 01 01 XX AA

示例2 — 批量DP：同时设置静音+音量+EQ模式

DP1: ID=01, type=bool, val=0x01

DP2: ID=02, type=value, len=4, val=15

DP3: ID=30, type=enum, len=1, val=0x02(Rock)

→ 5 bytes: 01 01 00 01 01

→ 8 bytes: 02 02 00 04 00 00 00 0F

→ 5 bytes: 30 04 00 01 02

数据长度 = 18 (0x12)

打包：55 01 06 00 12 01 01 00 01 01 02 02 00 04 00 00 00 0F 30 04 00 01 02 [校验] AA

下控应答 (针对 0x06):

55 03 06 00 01 [RESULT] XX AA

RESULT:

值	说明
0x00	全部成功
0x01	部分失败（首个失败DP ID跟在result后）

设计说明：静音控制有两个通道 — 快速直通(0x03)走简单帧，DP下发(0x06)走通用框架。快速通道适合高频简单操作，DP通道适合批量配置。两个通道最终操作的是同一个内部状态，互相同步。

5.7 DP状态上报 (0x07)

格式与 0x06 完全相同，但方向是下控→上控。

下控主动上报 — 静音状态变化：

```
55 03 07 00 05 01 01 00 01 01 [校验] AA
      ↑ DP ID=1, bool, len=1, val=1(静音)
```

下控主动上报 — 蓝牙连接状态变化：

```
55 03 07 00 05 04 04 00 01 03 [校验] AA
      ↑ DP ID=4, enum, len=1, val=3(播放中)
```

5.8 DP状态查询 (0x08)

上控请求全部DP：

```
55 01 08 00 00 [校验] AA      ← 数据长度=0，表示查询所有DP
```

上控按位选择查询：

数据字段为位图（每个bit对应一个DP ID），最大16字节（128个DP）：

```
55 01 08 00 01 AA [校验] AA      ← 查询DP 2,4,6,8 (bit1,3,5,7=1)
55 01 08 00 01 55 [校验] AA      ← 查询DP 1,3,5,7 (bit0,2,4,6=1)
```

下控应答：通过 0x07 命令上报对应DP点当前值。

5.9 查询设备信息-扩展 (0x09)

返回结构化二进制信息，比 0x01 的JSON更紧凑，适合资源受限场景。

上控→下控：

```
55 01 09 00 00 [校验] AA
```

下控→上控：

```
55 03 09 [LEN] [DATA] [校验] AA
```

DATA 格式（二进制TLV）：

[HW_MAJOR 1B]	[HW_MINOR 1B]	[HW_PATCH 1B]	← 硬件版本 3B
[SW_MAJOR 1B]	[SW_MINOR 1B]	[SW_PATCH 1B]	← 软件版本 3B
[DEVICE_TYPE 1B]			← 设备类型：0x01=音箱
[MAX_VOLUME 1B]			← 最大音量值
[FLASH_SIZE 4B BE]			← Flash大小（大端）
[RESERVED 4B]			← 保留

示例：

```
55 03 09 00 0C 01 00 00 01 00 01 01 1E 00 08 00 00 [校验] AA
                                     _____
                               hw=1.0.0  sw=1.0.1  type=音箱  max_vol=30  flash=512KB
```

5.10 OTA升级流程 (0x0A / 0x0B / 0x0C)

5.10.1 OTA开始 (0x0A)

上控→下控：

```
55 01 0A 00 05 [OTA_TYPE 1B] [FILE_SIZE 4B BE] [校验] AA
```

字段	字节	说明
OTA_TYPE	1	0x01 =语音文件, 0x02 =固件
FILE_SIZE	4	文件总字节数, 大端

下控→上控：

```
55 03 0A 00 02 [CHUNK_SIZE 1B] [RESULT 1B] [校验] AA
```

字段	字节	说明
CHUNK_SIZE	1	建议块大小编码: 0x01 =256B, 0x02 =512B, 0x03 =1024B
RESULT	1	0x00 =就绪, 0x01 =空间不足, 0x02 =类型不支持

示例 — 语音OTA 108KB：

```
上控：55 01 0A 00 05 01 00 01 B0 00 [校验] AA
      ↑ type=语音  size=110592 (0x1B000)

下控：55 03 0A 00 02 01 00 [校验] AA
      ↑ 256B/块, 就绪
```

5.10.2 OTA数据传输 (0x0B)

上控发送：

```
55 01 0B [LEN] [OFFSET 4B BE] [XOR 1B] [DATA NB] [校验] AA
```

字段	字节	说明
OFFSET	4	数据在文件中的偏移量, 大端
XOR	1	DATA的逐字节XOR校验

字段	字节	说明
DATA	N	数据块（通常256B）

下控应答（成功）：

```
55 03 0B 00 04 [OFFSET 4B BE] [校验] AA
```

应答中的OFFSET返回已成功写入的偏移（用于续传确认）。

下控应答（失败）：

```
55 03 0B 00 04 [OFFSET 4B BE] [校验] AA ← XOR校验失败，返回出错偏移，请重发
```

设计差异说明：MRK原协议OTA应答只回 0B 00 00，不区分成功/失败，靠上控超时判断。新协议改为应答出错偏移，上控可直接定位重发，效率更高。

5.10.3 OTA完成 (0x0C)

上控→下控：

```
55 01 0C 00 04 [CRC16 2B BE] [TOTAL_SIZE 2B BE] [校验] AA
```

字段	字节	说明
CRC16	2	整包CRC16-CCITT (多项式 0x1021, init=0)，大端
TOTAL_SIZE	2	实际写入总字节数的低16位

下控→上控：

```
55 03 0C 00 01 [RESULT] [校验] AA
```

RESULT	说明
0x00	校验通过，升级成功 ✓ (设备将自动复位)
0x01	CRC16不匹配，升级失败
0x02	写入字节数与TOTAL_SIZE不符

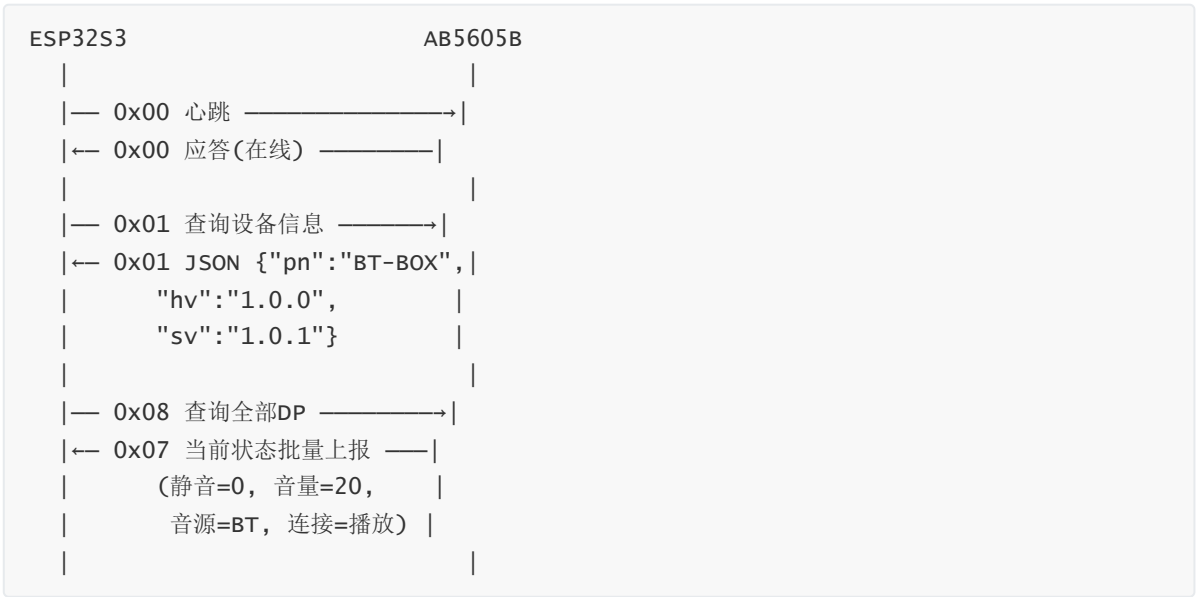
示例 — 升级成功：

```
上控：55 01 0C 00 04 A3 F2 00 05 [校验] AA
      CRC16=0xA3F2, total_size 低16位=5

下控：55 03 0C 00 01 00 [校验] AA
      result=0x00 成功 → 200ms后自动复位
```

六、通信流程示例

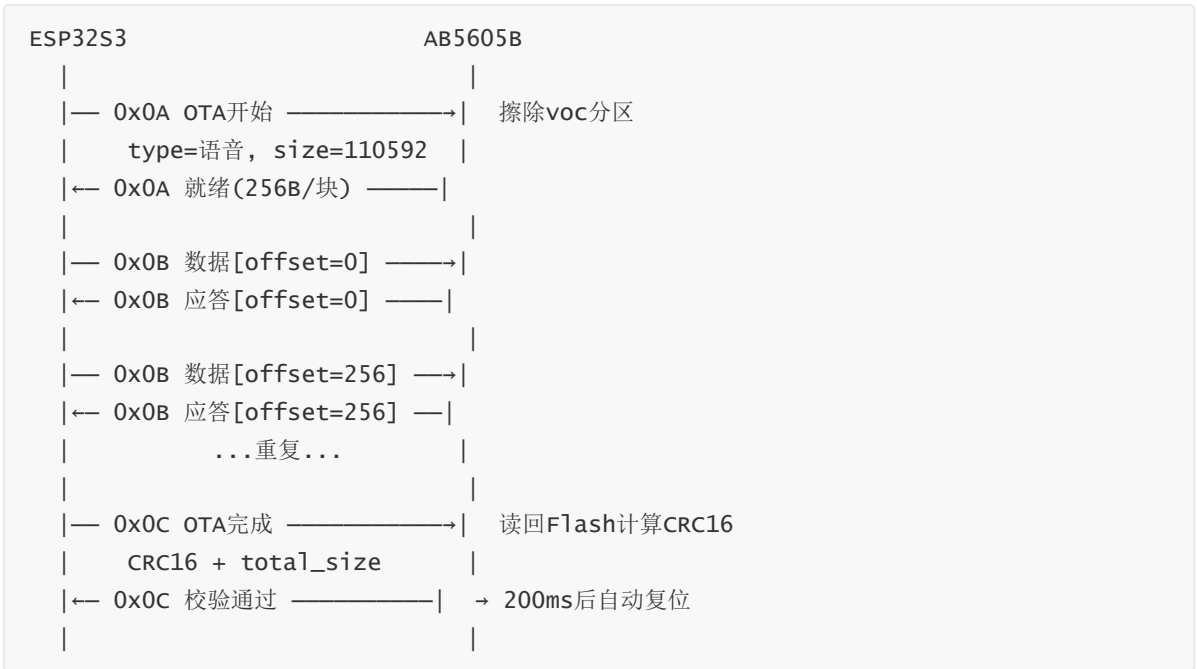
6.1 ESP32S3 启动 → 发现AB5605B → 获取信息



6.2 静音控制



6.3 语音OTA升级



6.4 下控主动上报（蓝牙连接状态变化）



七、与现有协议的兼容关系

当前 `func_mp3_uart_upd.c` 实现的简单协议和本协议的关系：

功能	当前协议 (简单)	新协议 (MRK兼容)	迁移策略
帧格式	[55][AA][CMD] [DATA]	[55][VER][CMD][LEN][DATA] [CKSUM][AA]	逐步迁移
帧头	55 AA (2B)	55 (1B) + 版本字	区分不同命令集
语音OTA ERASE	CMD=0x01	CMD=0x0A	新协议用0x0A
语音OTA DATA	CMD=0x02 (seq+data)	CMD=0x0B (offset+data)	新协议用0x0B
语音OTA FINISH	CMD=0x03	CMD=0x0C	新协议用0x0C
ACK格式	[AA][55][CMD] [RESULT]	同命令格式(下控版本=0x03)	统一为上/下控 帧格式
静音控制	无	CMD=0x03 或 DP 0x06	新增
版本查询	无	CMD=0x01	新增

共存方案

AB5605B 上同时工作两套协议解码：

- 检测到 [55 AA] 双字节帧头 → 走当前简单协议（命令字 0x01~0x03）
- 检测到 [55 01] 或 [55 03] + 命令字不在 [0x01,0x02,0x03] → 走新协议

两者可以共存，互不干扰。

八、错误处理

8.1 帧级错误

错误类型	检测方式	处理
帧头不匹配	等待 0x55	丢弃非帧头字节
长度超限	数据长度 > 缓冲区大小	丢帧，从下一帧头重新同步
校验和不匹配	计算checksum ≠ 接收值	丢帧，无应答（上控超时重发）
帧尾不匹配	帧尾 ≠ 0xAA	丢帧

8.2 业务级错误

错误码	说明	适用命令
0x00	成功	所有
0x01	不支持/失败	0x02, 0x0A, 0x06
0x02	参数超出范围	0x04
0x03	当前状态不允许	0x02 (忙时不允许复位)
0x10	DP ID不存在	0x06
0x11	DP类型不匹配	0x06

8.3 重试策略

场景	重试次数	间隔	备注
心跳无响应	3次	1000ms	判定离线
普通命令无响应	3次	100ms	递增20ms
OTA数据无响应	5次	20ms	快速重试
OTA数据XOR失败	立即重发	0ms	下控返回错误偏移

九、实施建议

9.1 ESP32S3端 (上控)

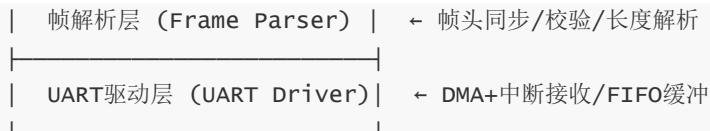
ESP32S3 建议实现层次：

应用层 (Business Logic)

DP编解码层 (DP Codec)

← 静音/音量/OTA业务逻辑

← DP单元打包/拆包



ESP32S3的UART有硬件FIFO（128字节），建议使用IDF的 `uart_driver_install()` 配置 `RX_BUFFER_SIZE=1024`。

9.2 AB5605B端 (下控)

当前 `func_mp3_uart_upd.c` 已具备：

- UART1 RX中断 + FIFO缓冲 ✓
- 帧头同步状态机 ✓
- Flash读写 ✓
- CRC16校验 ✓

需扩展：

- 长度字段解析（当前硬编码per-command）
- 校验和验证（当前无帧级校验）
- 帧尾验证
- DP点解码（新增 `0x06` / `0x07` / `0x08`）
- 新增命令处理器（静音、音量、版本查询）

9.3 命令优先级建议

1. 先实现快速直通命令（`0x03` 静音 / `0x04` 音量 / `0x01` 设备信息）— 改动最小
2. 再升级OTA协议（`0x0A→0x0B→0x0C` 替换 `0x01→0x02→0x03`）
3. 最后实现DP框架（`0x06` / `0x07` / `0x08`）— 为未来扩展做准备

9.4 附录A — 校验和计算参考代码

```
// C 校验和计算
uint8_t calc_checksum(const uint8_t *buf, uint16_t len) {
    uint8_t sum = 0;
    for (uint16_t i = 0; i < len; i++) {
        sum += buf[i];
    }
    return sum;
}

// 发送帧构造
void send_frame(uint8_t version, uint8_t cmd, const uint8_t *data, uint16_t data_len) {
    uint8_t frame[256];
    uint16_t idx = 0;

    frame[idx++] = 0x55;           // 帧头
    frame[idx++] = version;       // 版本
    frame[idx++] = cmd;           // 命令字
```

```

frame[idx++] = (data_len >> 8) & 0xFF; // 长度高字节
frame[idx++] = data_len & 0xFF;      // 长度低字节

for (uint16_t i = 0; i < data_len; i++) {
    frame[idx++] = data[i]; // 数据
}

frame[idx] = calc_checksum(frame, idx); // 校验和
idx++;

frame[idx++] = 0xAA; // 帧尾

uart_send(frame, idx);
}

```

9.5 附录B — Python工具参考

```

# Python 帧构造参考
import struct

def calc_checksum(data: bytes) -> int:
    return sum(data) & 0xFF

def build_frame(version: int, cmd: int, payload: bytes = b'') -> bytes:
    frame = bytearray()
    frame.append(0x55) # 帧头
    frame.append(version) # 版本
    frame.append(cmd) # 命令字
    frame.extend(struct.pack(">H", len(payload))) # 数据长度 (大端)
    frame.extend(payload) # 数据
    ck = calc_checksum(bytes(frame)) # 校验和
    frame.append(ck)
    frame.append(0xAA) # 帧尾
    return bytes(frame)

def parse_frame(data: bytes) -> dict:
    if len(data) < 7:
        return {"error": "帧长度不足"}
    if data[0] != 0x55:
        return {"error": "帧头不匹配"}
    if data[-1] != 0xAA:
        return {"error": "帧尾不匹配"}

    version = data[1]
    cmd = data[2]
    payload_len = struct.unpack(">H", data[3:5])[0]
    payload = data[5:5 + payload_len]
    checksum = data[5 + payload_len]

    if checksum != calc_checksum(data[:5 + payload_len]):
        return {"error": "校验和不匹配"}

    return {
        "version": version,
        "cmd": cmd,
    }

```



```
"payload_len": payload_len,  
"payload": payload,  
}
```

文档结束